

CPE 4750: Introduction to IoT System, Spring 2025

Final Team Project Report

IoT in Smart Building

Ethan Karnes & Margarita Marquez

Date: May 3, 2025

Electrical and Computer Engineering

Kennesaw State University

Faculty: Dr. Jeffrey L Yiin

Table of Contents

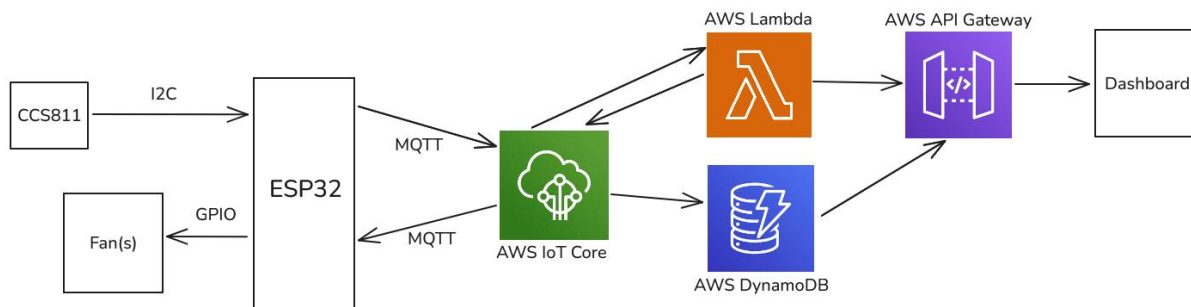
Project Background:.....	2
Project Goal:	2
Required Equipment:	2
Procedure:	3
1) PART 1; INITIALIZATION OF SETUP	3
2) PART 2; AWS THING SETUP	7
3) PART 3; ARDUINO CODE IMPLEMENTATION	11
4) PART 4; AWS RULE IMPLEMENTATION	13
5) PART 5; DASHBOARD IMPLEMENTATION	23
Source Code & Documentation:	28
Lessons Learned & Reflection:	28

Project Background:

The indoor air quality monitoring industry severely lacks affordable products to measure the level of volatile organic compounds (TVOCs). The current industry product's price ranges from \$100-\$2000. This creates an issue of scalability, particularly for consumers or organizations that need to monitor VOC levels across large buildings with multiple rooms. We intended to create something to fill this gap in the market.

Project Goal:

Our proposed solution is to use an Inland CCS811 gas sensor (to measure the Total Volatile Organic Compound) to collect air quality data. This would be interfacing with an ESP32-wroom-32 via I2C. The ESP32 would be connected to the internet using Wi-Fi and the data would be transmitted to a web service provider such as Amazon Web Service. With these connections it would be possible to view the sensors output in real time and adjust alerts and notifications. Once the TVOC's reach above a specific threshold, miniature fans will turn on. All of the components and connections are very affordable, bringing indoor air quality monitoring to those without large budgets.



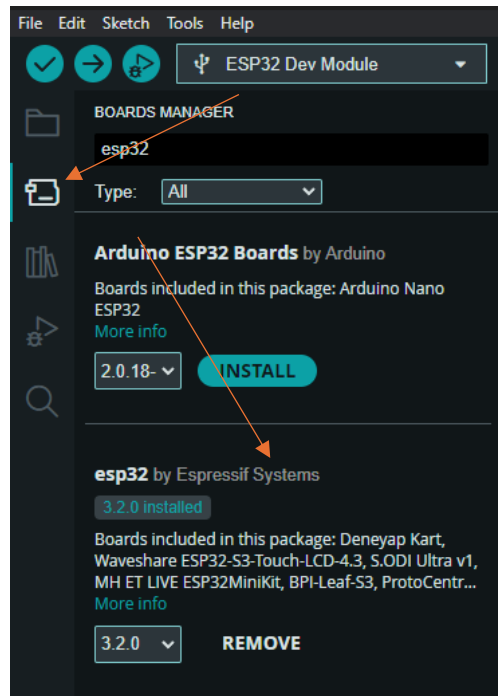
Required Equipment:

- Personal computer (Windows/Mac OS, Instructions are for Windows)
- HiLetgo ESP32-WROOM-32 WiFi + BLE Microcontroller
- Micro USB cable and Jumper Wires (female-to-female)
- CCS811 gas sensor
- AWS Account
- Arduino IDE Program
- 30mm Fan(s)

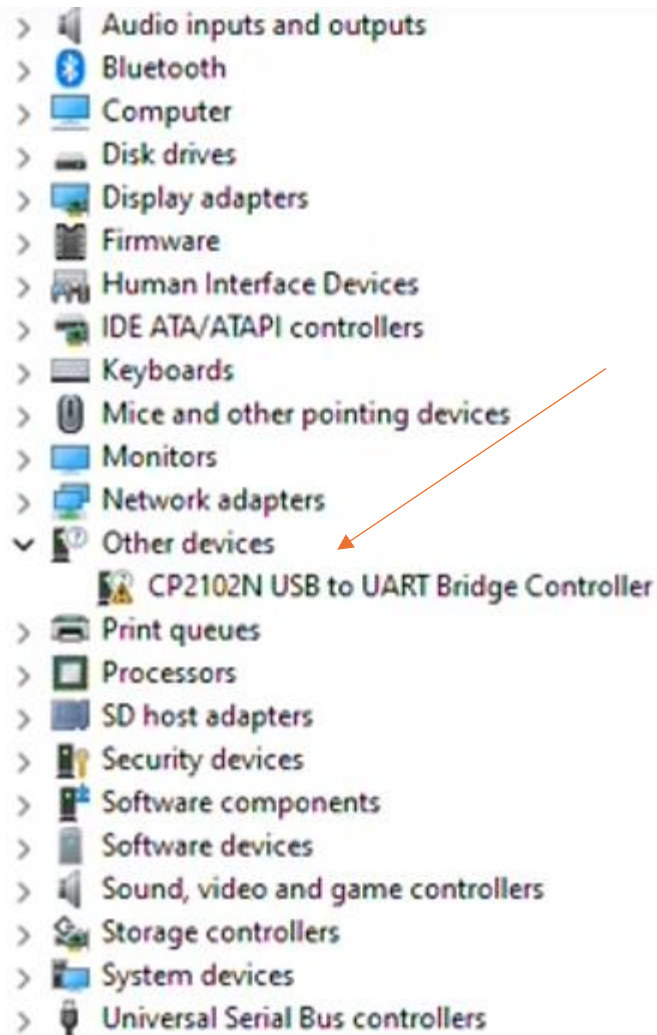
Procedure:

1) **PART 1; INITIALIZATION OF SETUP.** Setting up the physical circuit and serial communication.

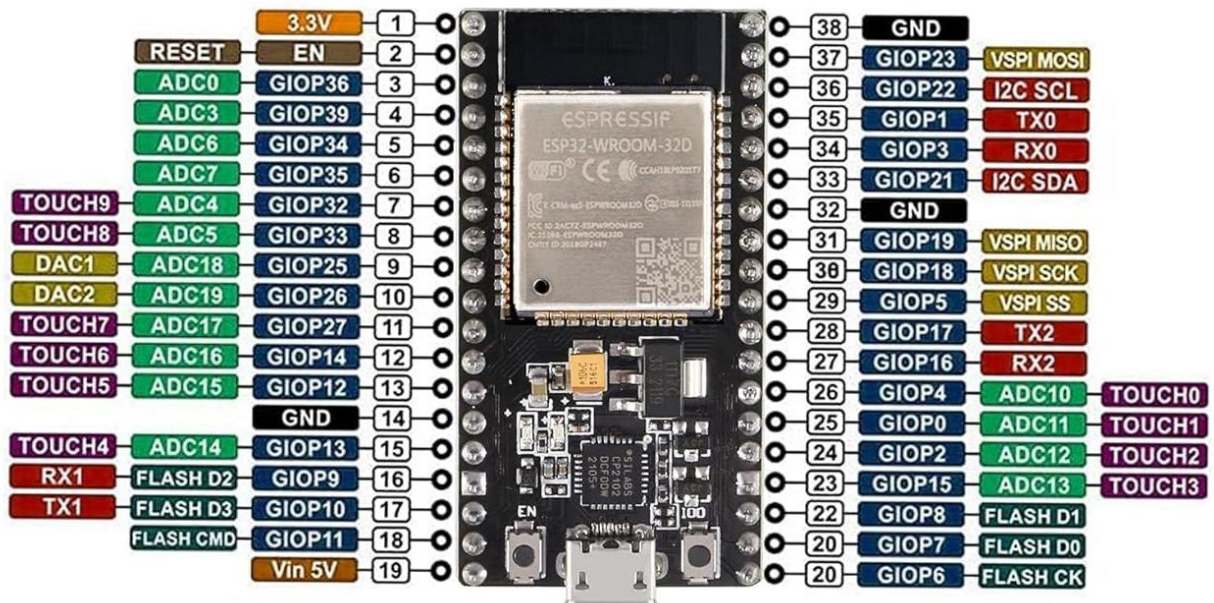
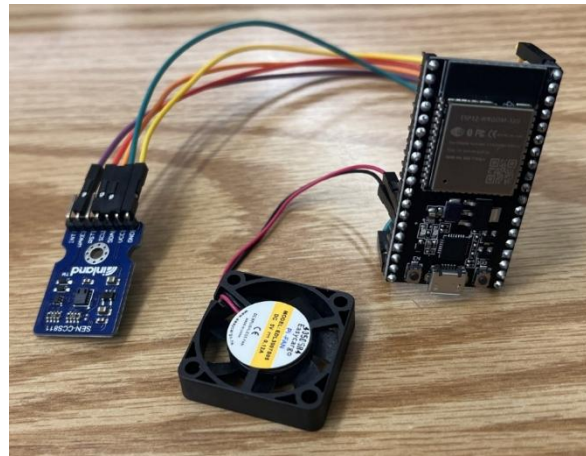
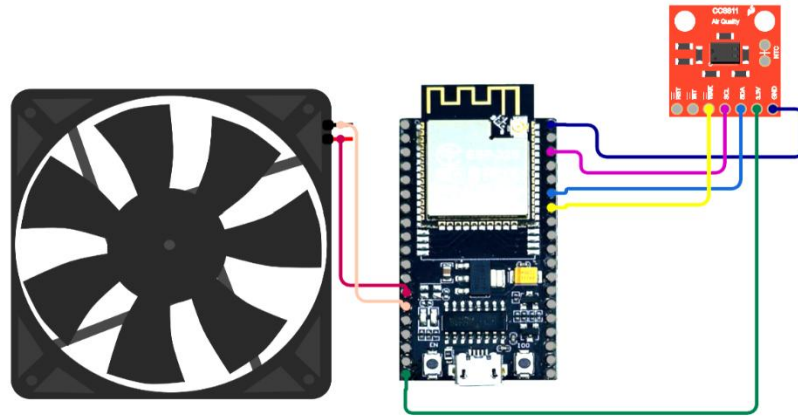
- a) Using the micro-USB cable, plug the ESP32 in to a personal computer. A red light should glow on the board indicating the ESP32 is powered.
- b) Download and launch the Arduino IDE and on the sidebar navigate to **Boards Manager** and search for “esp32.” Install the “esp32 by Espressif.”



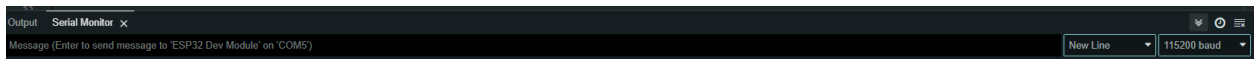
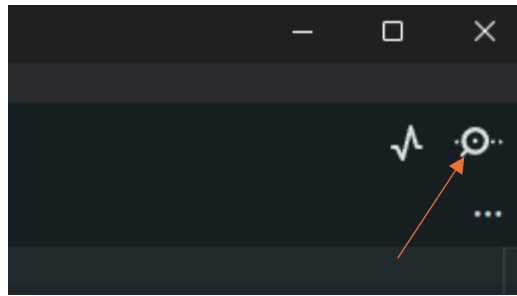
- c) Now navigate to the **Library Manager**. Install the packages “Adafruit CCS811 Library”, “ArduinoJson by Benoit”, “PubSubClient by Nick O’leary”. Download all dependencies that these need too, but the Arduino IDE will most likely automatically download them.
- d) With all the dependencies downloaded for the Arduino, we lastly need to download one more thing that allows communication between the ESP32 and the computer itself. Go to this website: <https://www.silabs.com/developer-tools/usb-to-uart-bridge-vcp-drivers?tab=downloads> and navigate to the downloads tab. Download the CP210x Universal Windows Driver and unzip it. Open Device Manager on windows and navigate to **Other Devices** as seen in the image below. Right click on the USB to UART Bridge Controller and click update drivers. Click “browse my computer for drivers” and navigate to the unzipped universal driver folder. Do not click anything inside of the folder, so your location should be \placeholder\CP210X_Universal_Windows_Driver. Click next and it should automatically update the drivers.



- e) Restart your computer and now when you plug the ESP32 in and launch the Arduino IDE, navigate to the USB symbol next to the debug play button in the top left and it should say “select board and port.” Search for “ESP32 Dev Module” and the port that it is recognized on and now you should be able to upload c++ programs onto the ESP32. Whenever you plug the ESP32 into the computer from now on, in the serial monitor, it should tell you that the ESP32 is connected.
- f) Next is getting the sensor and the fan connected to the ESP32 itself. In figure 2 you can see the cable configuration. Starting with the sensor we are not using the INT and RST pins. For the other pins connect WAKE -> GND, SCL -> SCL (Pin 36), SDA -> SDA (Pin 33), VCC -> 5V (Pin 19), and GND -> GND. For pin numbers, follow figure 4 which shows the pin numbers visually too. The fan only connect to two pins and for our program it is the red cable -> Pin 13 (GPIO12) and black -> GND. That should be all for the wiring!



- g) Lastly, we should check if the wiring is correct by using a test program with the ESP32. Using the SensorTesting.ino code from the GitHub, paste it into the Arduino IDE and save the sketch by pressing ctrl+s. While having the ESP32 plugged in and recognized in the serial monitor, press the arrow in the top left corner of the Arduino IDE.



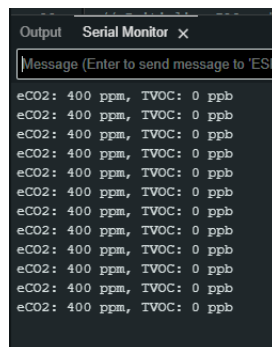
This will start to compile the program. It will take a few seconds, but . It will also open the output on the bottom part of the screen. When it is done compiling, it will start to search for a device. In the image below, the “Conecting.....” denotes the Arudino IDE is searching for the ESP32.

```
Serial port COM5
Connecting.....
Chip is ESP32-D0WD-V3 (revision v3.1)
Features: WiFi, BT, Dual Core, 240MHz, VRef calibration in efuse, Coding Scheme None
Crystal is 40MHz
MAC: 5c:01:3b:74:77:64
Uploading stub...
Running stub...
Stub running...
Changing baud rate to 921600
Changed.
Configuring flash size...
Flash will be erased from 0x00001000 to 0x00007fff...
Flash will be erased from 0x00008000 to 0x00008fff...
Flash will be erased from 0x0000e000 to 0x0000ffff...
Flash will be erased from 0x00010000 to 0x0005ffff...
Compressed 24976 bytes to 15972...
```

On the ESP32 device itself, physically, press the **EN** button next to the USB port on the bottom to set the ESP32 into “flash” mode. It may take a few tries to get used to this which is perfectly fine.

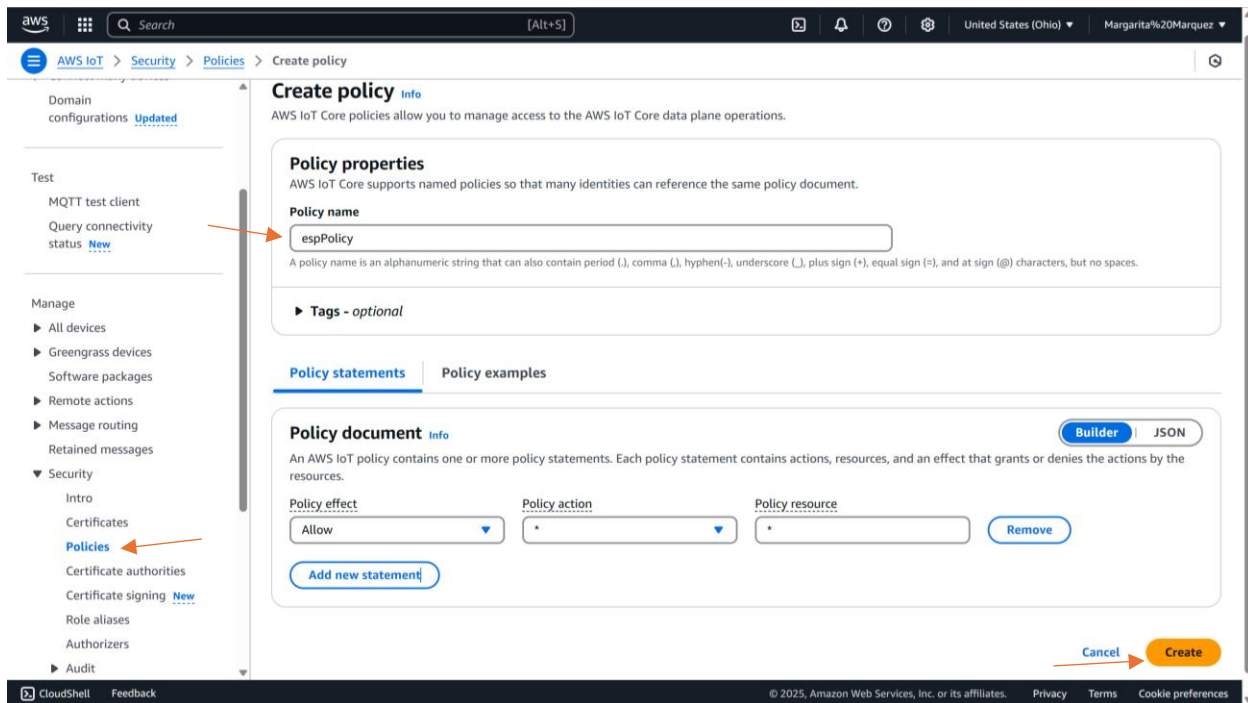


When the ESP32 gets flashed with the program, it will say “Leaving...” in the output console. This signifies that the ESP32 was flashed successfully. Wait about 10 seconds after you see this and then press the **EN** button again to reboot the ESP32. In a few seconds after resetting it, open the serial monitor and check if the eCO2 and TVOC values are read.

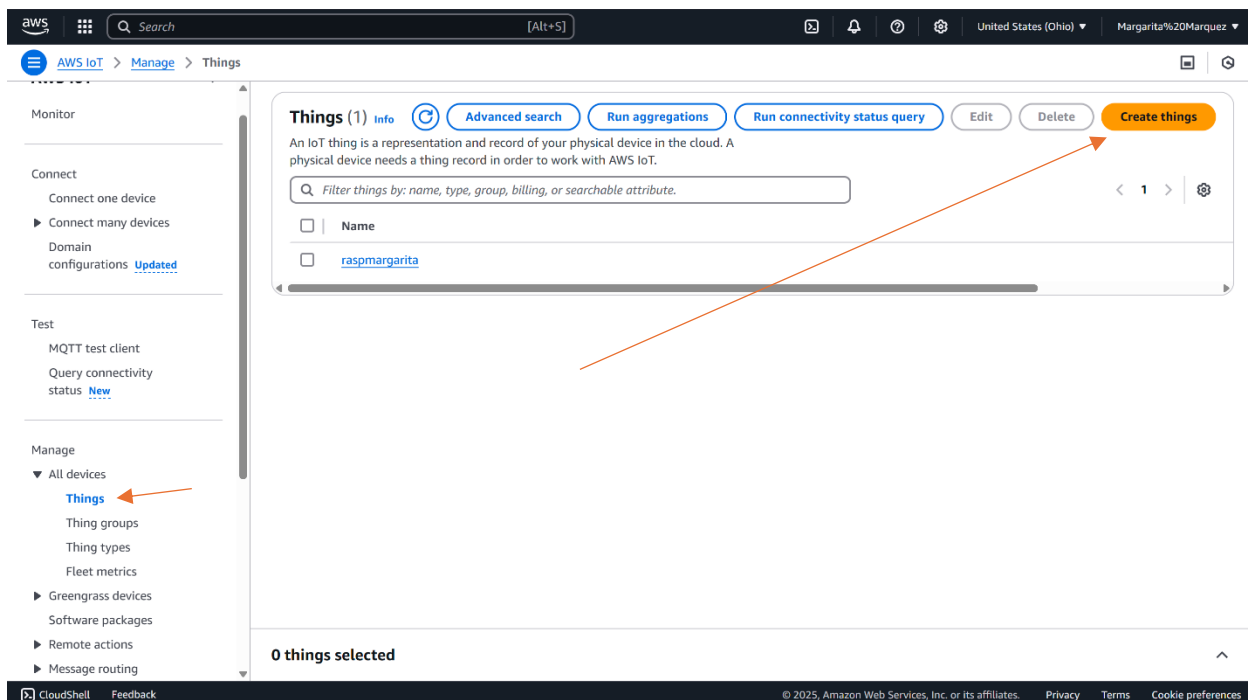


2) PART 2; AWS THING SETUP. For this assignment, we will be using MQTT Protocol through AWS (Amazon Web Services) to publish the gas sensor’s data to AWS IoT Core.

- a) First, go to <https://aws.amazon.com/> and create a free tier AWS account. However, it's worth noting that after the first 12 months, they will begin charging you.
- b) Once you finish setting up your AWS account, click on the services in the top right corner. From there, scroll down until you see **Internet of Things**. Then under the Internet of Things tab, click **IoT Core**.
- c) In the AWS IoT core, go to **Security -> Policies**. Then on the top right, click on the orange “**Create policy**”.



- d) In the AWS IoT Core, go to **Manage -> All Devices -> Things**. From there, go to the orange button that says **“Create things”**. In the given options, go to **Create single thing** and click next.

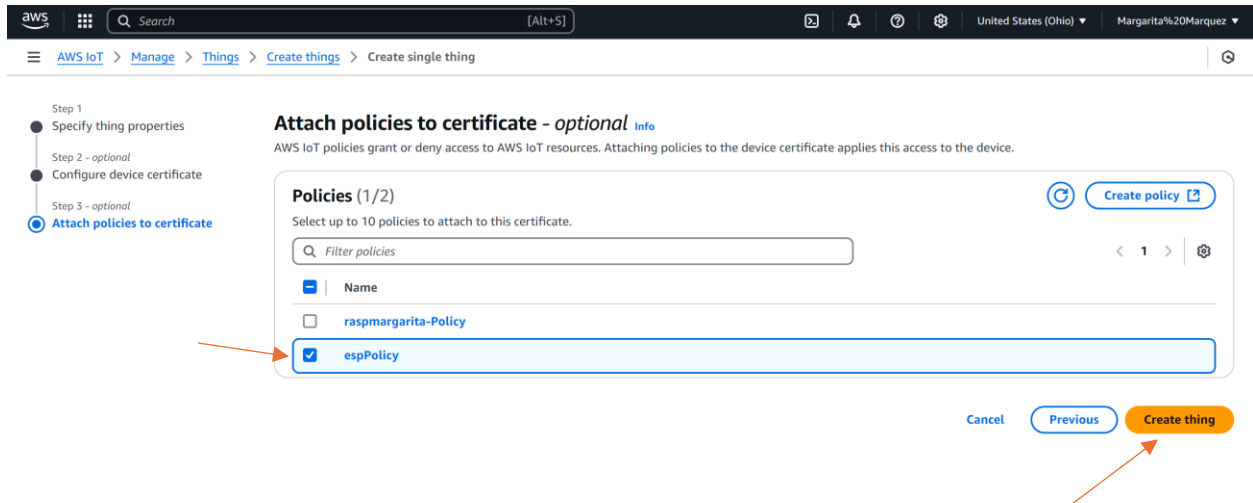


- e) Once you have your policy create, go to **Manage -> Things -> Create Things**.

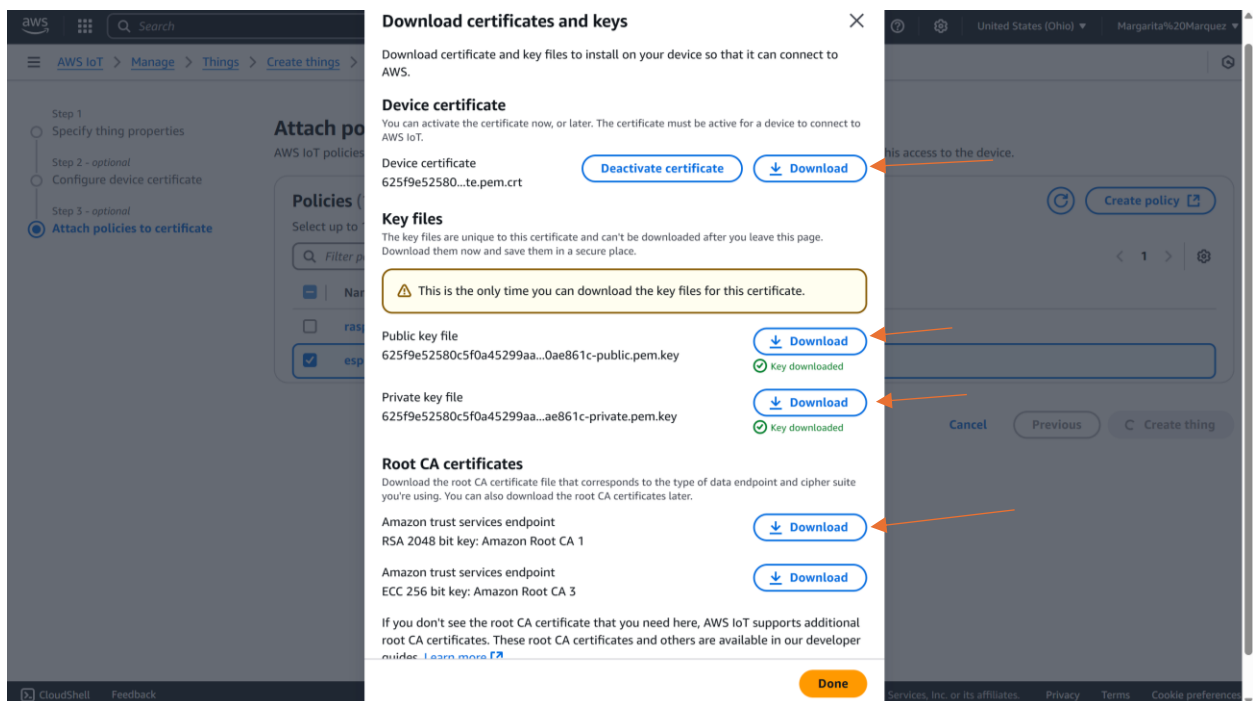
- f) Click **Create Single Thing** since we are only going to be working with one ESP32 for this project.
- g) Name the Thing to the name of the hardware. For this project, “ESP32” will do fine. For now, put “**No shadow**” for the Device Shadow section on the bottom.

The screenshot shows the AWS IoT console interface for creating a single thing. The breadcrumb navigation at the top indicates the path: AWS IoT > Manage > Things > Create things > Create single thing. On the left, a sidebar shows the progress: Step 3 - optional, Attach policies to certificate. The main content area is divided into sections. The 'Thing properties' section has a 'Thing name' field containing 'ESP32', with a note below it: 'Enter a unique name containing only: letters, numbers, hyphens, colons, or underscores. A thing name can't contain any spaces.' Below this is the 'Additional configurations' section, which includes expandable options for 'Thing type - optional', 'Searchable thing attributes - optional', 'Thing groups - optional', 'Billing group - optional', and 'Packages and versions - optional'. The 'Device Shadow' section at the bottom has three radio button options: 'No shadow' (which is selected), 'Named shadow' (with a sub-note: 'Create multiple shadows with different names to manage access to properties, and logically group your devices properties.'), and 'Unnamed shadow (classic)' (with a sub-note: 'A thing can have only one unnamed shadow.'). The footer of the console shows 'CloudShell', 'Feedback', and copyright information for Amazon Web Services, Inc. or its affiliates, along with links for 'Privacy', 'Terms', and 'Cookie preferences'.

- h) After creating a single thing, the AWS is going to ask which policy should be attached to your thing. We already have created a policy for this in the beginning of the project. Thus, selected the newly made policy and go to **Create Thing**



- i) Make sure to download all the certificates except the Amazon Root CA 3 as it's not necessary for this project. This will be the ONLY time you will be able to download all the certificates.



3) PART 3; ARDUINO CODE IMPLEMENTATION. This section will focus on implementing the main program to get MQTT messages publishing to AWS. This will run through how to get the code to work using your setup.

- a) Download the ESP32MQTTPublishing.ino program from the GitHub. Reopen the Arduino IDE program and insert the given code. In line 10 and 11, insert your wifi's ssid and password.

```
9 // Wi-Fi Configuration
10 const char* ssid = "Enter Wifi ssid";
11 const char* password = "Enter Wifi Password";
```

- b) In line 14, enter your IoT Core Configuration. You can find it by going to AWS IoT Core and click Domain configuration in the left menu. Your AWS account has a device data endpoint to connect to AWS. Copy the Domain name. It has the form: "youramazondomain.amazonaws.com"
- c) Using the downloaded certificate keys, replace line 21 with your Things certificate. In your files, hover over the file that ends with -certificate.pem. Open the file with word editor and copy everything between the lines saying "BEGIN CERTIFICATE" and "END CERTIFICATE". Do the same with line 27 with your private key and line 33 with you Amazon CA1 certificate.

	AmazonRootCA1 .pem	4/22/2025 7:07 PM	PEM File	2 KB
	-private.pem.key	4/22/2025 7:07 PM	KEY File	2 KB
	-public.pem.key	4/22/2025 7:07 PM	KEY File	1 KB
	-certificate.pem	4/22/2025 7:07 PM	Security Certificate	2 KB

- d) With the code fully filled out, flash it onto your ESP32 just how we did earlier with the sensor testing program.
- e) Go to **AWS IoT page -> Test -> MQTT** test client. In the Subscribe to Topic filter, type in "esp32" and click subscribe. Now press the EN button on the ESP32 (make sure you have everything plugged in and the code is filled out correctly) to get the program to start. You should see an output in the serial monitor similar to the image below. Make sure to set the baud rate to 115200 in the bottom left corner. (Grabbing the NTP time can take <20s sometimes)

MQTT test client [Info](#)

You can use the MQTT test client to monitor the MQTT messages being passed in your AWS account. Devices publish MQTT messages that are identified by topics to communicate their state to AWS IoT. AWS IoT also publishes MQTT messages to inform devices and apps of changes and events. You can subscribe to MQTT message topics and publish MQTT messages to topics by using the MQTT test client.

▶ **Connection details** ✔ Connected

Disconnect

To disconnect from the MQTT test client, choose Disconnect. To re-establish a connection, you can update the details and choose Connect.

Subscribe to a topic

Publish to a topic

Topic filter [Info](#)

The topic filter describes the topic(s) to which you want to subscribe. The topic filter can include MQTT wildcard characters.

esp32

▶ **Additional configuration**

Subscribe

Output Serial Monitor ✕

Not connected. Select a board and a port to connect automatically.

entry 0x400805b4
Connecting to WiFi.....
Connected!
Waiting for NTP time
Time acquired!
Connecting to AWS IoT
AWS Connected!
Published:
{ "eco2":400,"tvoc":0,"timestamp":"2025-04-24 18:55:06","deviceId":"esp32"}
Published:
{ "eco2":400,"tvoc":0,"timestamp":"2025-04-24 18:55:16","deviceId":"esp32"}

▼ esp32

April 24, 2025, 18:55:17 (UTC-0400)

```
{
  "eco2": 400,
  "tvoc": 0,
  "timestamp": "2025-04-24 18:55:16",
  "deviceId": "esp32"
}
```

▶ **Properties**

4) PART 4; AWS RULE IMPLEMENTATION. Now we need to make the Rules for the project. This is how data is routed around the AWS IoT Core and other architecture.

- a) For the first rule, go to **AWS IoT-> Manage -> Message Routing -> Rules -> Create rule**. In the Rule name, put ESP32DynamoRule in the rule name then click **Next**. In the SQL Statement put **SELECT * FROM 'esp32'** then click **Next**. In Action 1, open the list and select **DynamoDBv2**. Then right click **Create DynamoDB table** to open on a new page. For the table name, type “ESP32-Data”. Copy the information for the Partition key and Sort key that’s in the figure below. Then click Next and it should take a couple seconds to create the table. Go back to the separate tab with the rules and click the refresh arrow button so that you can select your newly created table.

The screenshot shows the AWS IAM console 'Create table' page. The breadcrumb navigation is 'DynamoDB > Tables > Create table'. The page title is 'Create table'. Below the title, there is a 'Table details' section with an 'Info' link. It states: 'DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.' The 'Table name' field is 'ESP32-Data'. Below it, a note says: 'This will be used to identify your table.' and 'Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.)'. The 'Partition key' section shows 'deviceId' as the key and 'String' as the data type. Below it, a note says: 'The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.' and '1 to 255 characters and case sensitive.' The 'Sort key - optional' section shows 'timestamp' as the key and 'String' as the data type. Below it, a note says: 'You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.' and '1 to 255 characters and case sensitive.' The 'Table settings' section has two options: 'Default settings' (selected) and 'Customize settings'. The 'Default settings' option has a description: 'The fastest way to create your table. You can modify most of these settings after your table has been created. To modify these settings now, choose 'Customize settings'.'

Create a new IAMRole and name it ESP32DynamoRole. Then finally create the rule. Make sure your newly created rule matches the following figures.

Edit rule: ESP32DynamoRule

Rule properties

Rule description - optional

Enter a description to provide additional details about the rule to others.

A description of your new rule

SQL statement [Info](#)

SQL version

The version of the SQL rules engine to use when evaluating the rule.

2016-03-23

SQL statement

Enter a SQL statement using the following: `SELECT <Attribute> FROM <Topic Filter> WHERE <Condition>`. For example: `SELECT temperature FROM`
learn more, see [AWS IoT SQL Reference](#).

1 `SELECT * FROM 'esp32'`

Rule actions [Info](#)

Select one or more actions to happen when the above rule is matched by an inbound message. Actions define additional activities that occur when messages arrive, like storing them in a database, invoking cloud functions, or sending notifications. You can add up to 10 actions.

Action 1

DynamoDBv2 [Remove](#)
Split message into multiple columns of a DynamoDB table (DynamoDBv2)

Table name [Info](#)

ESP32-Data



[Create DynamoDB table](#)

IAM role

Choose a role to grant AWS IoT access to your endpoint.

ESP32DynamoRole



[View](#)

[Create new role](#)

AWS IoT will automatically create a policy with a prefix of "aws-iot-rule" under your IAM role selected.

[Add rule action](#)

Error action - optional

You can optionally set an action that will be executed when something goes wrong with processing your rule. If two rule actions in the same rule fail, the error action receives one message that contains both errors.

[Add error action](#)

To test if the table works, plug the ESP32 back in and wait for it to start sending messages to AWS using MQTT. When the messages are received by AWS, they should start to show up in the table as shown below. Navigate to the DynamoDB section of AWS. Click on the **Explore Items** section in the sidebar and click on the ESP32-Data table. It should now contain timestamped values of the gas sensors data.

ESP32-Data

▼ Scan or query items

☒ Scan ☐ Query

Select a table or index Select attri

Table - ESP32-Data All attri

► Filters - optional

Run Reset

⚠ Stopped - Items returned: 50 - Items scanned: 50 - Efficiency: 100% - RCUs consumed: 1

Table: ESP32-Data - Items returned (50)

Scan started on April 25, 2025, 14:53:41

☐	deviceId (String)	timestamp (String)	eco2	tvoc
☐	esp32	2025-04-24 21:11:09	403	0
☐	esp32	2025-04-24 21:11:19	400	0
☐	esp32	2025-04-24 21:11:29	437	5
☐	esp32	2025-04-24 21:11:39	439	5

b) For the second rule we need to create a simple device shadow and its rule. In our case the device shadow lets us control GPIO pins without ever having to physically mess with the ESP32, letting it be completely AWS based. Head to **AWS IoT -> Manage -> All devices -> Things**. Click on **ESP32** and click on **Device Shadows**. Click on **Create Device Shadow** and create an Unnamed (classic) Shadow.

ESP32 Connectivity indexing is not enabled Info Create secure tunnel Edit Delete

Thing details

Name
ESP32

ARN
[arn:aws:iotus-east-2:942709711968:thing/ESP32](#)

Type
-

Billing group
-

Attributes | Certificates | Thing groups | Command history | **Device Shadows** | Activity | Packages and versions | Jobs | Alarms | Defender metrics

Attributes (0) Info

Attributes are key-value pairs that can be searchable or non-searchable. Searchable attributes can be used to filter lists of things without using fleet indexing. Non-searchable attributes can be used to find things, but only when fleet indexing is turned on.

Key	Value	Type
No attributes		

There are no attributes to display.

With the shadow created, now go to **AWS IoT-> Manage -> Message Routing -> Rules -> Create rule**. In the Rule name, put **ESP32ShadowRule** in the rule name then click **Next**. In the SQL Statement put **SELECT tvoc AS state.reported.tvoc FROM 'esp32'** then click **Next**. In Action 1, open the list and select **Republish to AWS IoT**. In the Topic, put **\$\$\$aws/things/ESP32/shadow/update**. For the Quality of service, leave it at 0. Create a new IAMRole and name it **ESP32ShadowRole**.

Edit rule: ESP32ShadowRule

Rule properties

Rule description - optional

Enter a description to provide additional details about the rule to others.

A description of your new rule

SQL statement [Info](#)

SQL version

The version of the SQL rules engine to use when evaluating the rule.

2016-03-23

SQL statement

Enter a SQL statement using the following: SELECT <Attribute> FROM <Topic Filter> WHERE <Condition>. For example: SELECT temperature FROM 'esp32' WHERE temperature > 100. For more information, see [AWS IoT SQL Reference](#).

1 SELECT tvoc AS state.reported.tvoc FROM 'esp32'

Rule actions [Info](#)

Select one or more actions to happen when the above rule is matched by an inbound message. Actions define additional activities that occur when messages arrive, like storing them in a database, invoking cloud functions, or sending notifications. You can add up to 10 actions.

Action 1

Republish to AWS IoT topic
Republish a message to an AWS IoT topic

Remove

Topic [Info](#)

\$\$aws/things/ESP32/shadow/update

Quality of service

When subscribing to a topic, quality of service 0 is chosen by default.

- ☒ 0 - The message is delivered at most once
- ☐ 1 - The message is delivered at least once

MQTT 5 properties [Info](#)

Add MQTT 5 properties by choosing a property type and adding a value.

Add property

You can add 5 more properties.

User properties [Info](#)

Add user properties in the MQTT header.

Add user property

IAM role

Choose a role to grant AWS IoT access to your endpoint.

ESP32ShadowRole



View

Create new role

Click on the **View** button next to the newly created IAM role. Click on **Add permissions -> Attach policies**. In the menu, search for **AWSIoTFullAccess**. This permission will give the rule the ability to publish to the shadow.

[Permissions](#) | [Trust relationships](#) | [Tags](#) | [Last Accessed](#) | [Revoke sessions](#)

Permissions policies (2) [Info](#)

You can attach up to 10 managed policies.

[Simulate](#) [Remove](#) [Add permissions](#)

Filter by Type: All types

☐	Policy name	Type	Attached entities
☐	aws-iot-rule-ESP32ShadowRule-action-1-role-ESP32ShadowRole	Customer managed	1
☐	AWSIoTFullAccess	AWS managed	2

Go back to the rule creation tab and click **Next**. On the last screen, click create and this rule is now setup. Before finished with the shadow completely we need to modify the Device Shadow Document. Navigate back to the ESP32 Device Shadows menu and click on your **Classic Shadow**. Scroll down and you should see the Device Shadow document. Click **Edit** and modify the **desired state** to “ventilation”: 0. Do not modify the reported state. Do not worry if it says “welcome to aws” or something similar as that does not affect anything. Now the device shadow should work as intended.

Edit Device Shadow state

i Updating the Device Shadow state sends update messages to your apps and services.

Device Shadow state

```

1 {
2   "state": {
3     "desired": {
4       "ventilation": 0
5     },
6     "reported": {
7     }
8   }
9 }
    
```

- c) Go back to your rules menu so you can create the third rule. In the Rule name, put **TVOCNSNS** in the rule name then click **Next**. In the SQL Statement put **SELECT * FROM 'esp32' WHERE tvoc > 500** then click **Next**.

Rule properties

Rule description - optional

Enter a description to provide additional details about the rule to others.

A description of your new rule

SQL statement [Info](#)

SQL version

The version of the SQL rules engine to use when evaluating the rule.

2016-03-23

SQL statement

Enter a SQL statement using the following: SELECT <Attribute> FROM <Topic Filter> WHERE <Condition>. For example: SELECT temperature FROM 'iot/topic' WHERE

1 SELECT * FROM 'esp32' WHERE tvoc > 500

In Action 1, open the list and select **Simple Notification Service (SNS)**. Create SNS Topic and it'll take you to a new tab. Under the details tab, select the **Standard** option. Under the name, put HighTVOCAlert and create the topic.

Create topic

Details

Type | [Info](#)

Topic type cannot be modified after topic is created

☐ **FIFO (first-in, first-out)**

- Strictly-preserved message ordering
- Exactly-once message delivery
- Subscription protocols: SQS

☒ **Standard**

- Best-effort message ordering
- At-least once message delivery
- Subscription protocols: SQS, Lambda, Data Firehose, HTTP, SMS, email, mobile application endpoints

Name

Maximum 256 characters. Can include alphanumeric characters, hyphens (-) and underscores (_).

Display name - optional | [Info](#)

To use this topic with SMS subscriptions, enter a display name. Only the first 10 characters are displayed in an SMS message.

Maximum 100 characters.

After creating the topic, you need to subscribe to receive the email alerts. Click the **Create Subscription** button inside of your HighTVOCAlert topic.

HighTVOCAlert

EditDeletePublish message

Details

Name
HighTVOCAlert

ARN
arn:aws:sns:us-east-2:942709711968:HighTVOCAlert

Type
Standard

Display name
-

Topic owner
942709711968

SubscriptionsAccess policyData protection policyDelivery policy (HTTP/S)Delivery status loggingEncryptionTagsIntegrations

Subscriptions (1)

EditDeleteRequest confirmationConfirm subscriptionCreate subscription

ID

Endpoint

Status

Protocol

Inside of the create subscription menu, select **Email** in the protocol and enter your email address you want to receive the alerts in into the **Endpoint** box. You will need to go to your email inbox and confirm an email from AWS saying you would like to be notified of this subscription.

Create subscription

Details

Topic ARN

Protocol

The type of endpoint to subscribe

Endpoint

An email address that can receive notifications from Amazon SNS.

[Info](#) After your subscription is created, you must confirm it.

Go back to the rules page and refresh on SNS Topic until you see the newly created SNS Topic. Make the message format RAW. Create a new IAMRole and name it TVOCSNSAlert. Then finally create the rule. Make sure your newly created rule matches the figures.

Rule actions Info

Select one or more actions to happen when the above rule is matched by an inbound message. Actions define additional activities that occur when me functions, or sending notifications. You can add up to 10 actions.

Action 1

Simple Notification Service (SNS)

Send a message as an SNS push notification

arn:aws:sns:us-east-2:942709711968:HighTVOCAlert

↺

↻

Create SNS topic

SNS topic Info

arn:aws:sns:us-east-2:942709711968:HighTVOCAlert

Message format

RAW

IAM role

Choose a role to grant AWS IoT access to your endpoint.

TVOCSNSAlert

↺

↻

View

Create new role

AWS IoT will automatically create a policy with a prefix of "aws-iot-rule" under your IAM role selected.

Add rule action

- d) Now onto creating the last rule. Create a new rule as usual and in the rule name, type "VentilationAuto" in and click **Next**. In the SQL Statement put **SELECT * SELECT tvoc FROM 'esp32'** then click **Next**.

Configure SQL statement Info

Add a simplified SQL syntax to filter messages received on an MQTT topic and push the data elsewhere.

SQL statement Info

SQL version

The version of the SQL rules engine to use when evaluating the rule.

2016-03-23

SQL statement

Enter a SQL statement using the following: SELECT <Attribute> FROM <Topic Filter> WHERE <Condition>. For example: SELECT temperature FROM

1	SELECT * SELECT tvoc FROM 'esp32'
---	-----------------------------------

In Action 1, open the list and select **Lambda**. Create a Lambda function and it'll take you to a new tab.

Rule actions [Info](#)

Select one or more actions to happen when the above rule is matched by an inbound message. Actions define additional activities that o invoking cloud functions, or sending notifications. You can add up to 10 actions.

Action 1

▼ **Lambda** 
Send a message to a Lambda function

Lambda function [Info](#)

Choose Lambda function ▼   **Create a Lambda function** 

Lambda function version

Choose Lambda function version ▼ 

Add rule action

In this new tab, click **Author from scratch**. Change the Runtime language from **Node Js to Python**. Enter a function name of **AutoVentilation**, and architecture of **x86_64**. Scroll down and click create function.

Create function [Info](#)

Choose one of the following options to create your function.

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.
AutoVentilation
Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

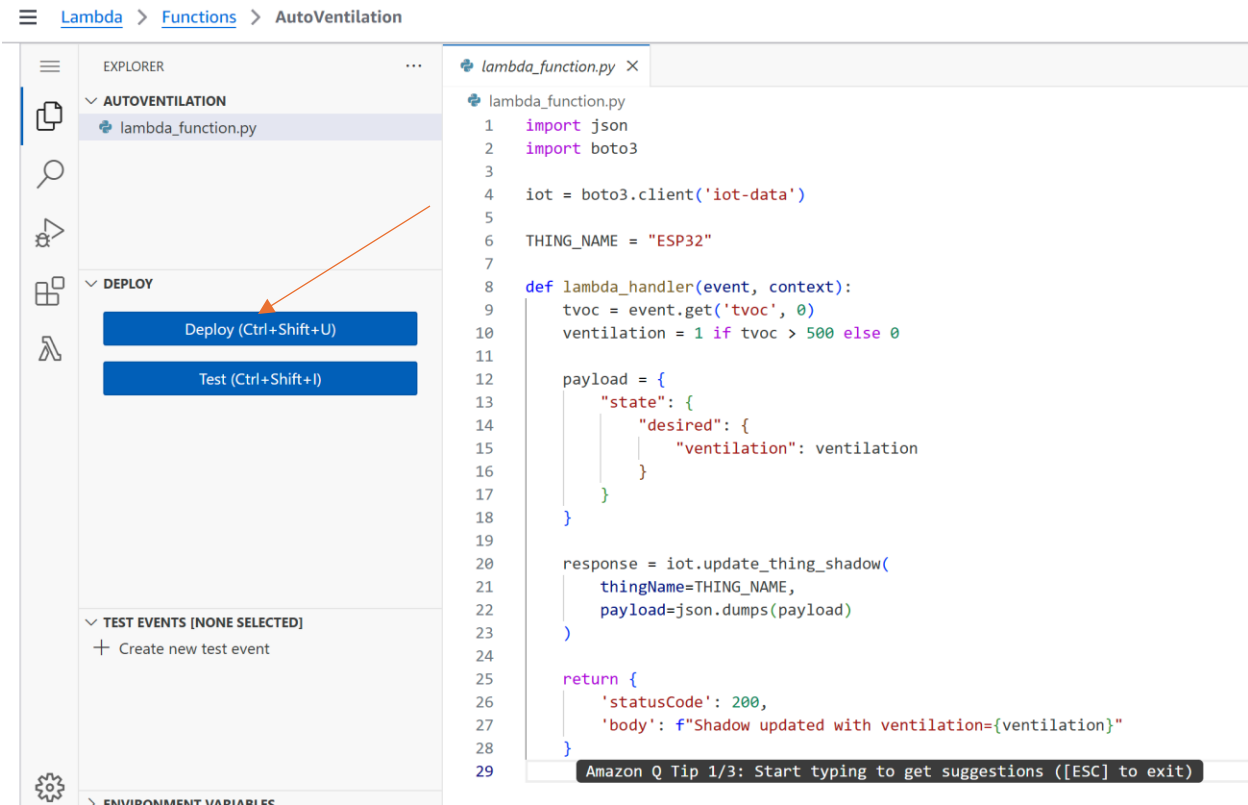
Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.
Python 3.13 

Architecture [Info](#)
Choose the instruction set architecture you want for your function code.
☒ x86_64
☐ arm64

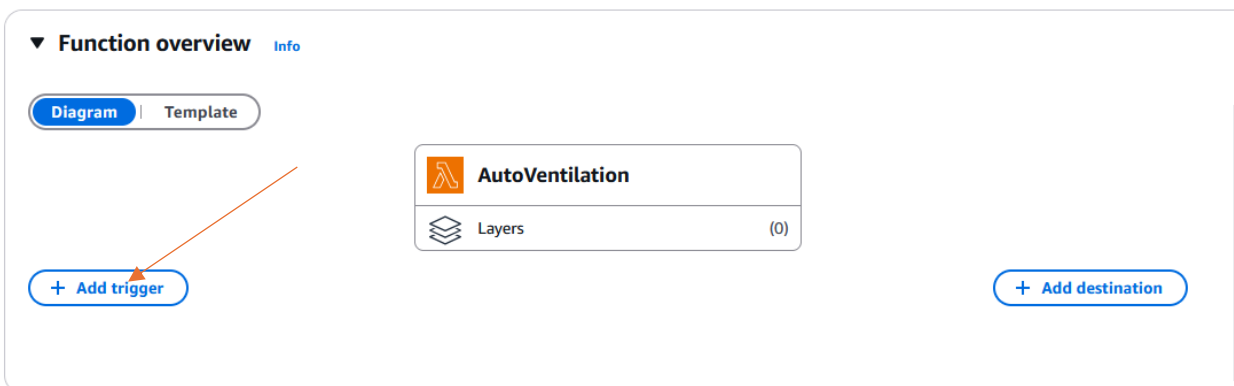
Permissions [Info](#)
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

► **Change default execution role**

This will bring you to the AWS Lambda homescreen, scroll down to the Lambda Function area and paste the AutoVentilationScript.py provided in the GitHub. Then click **Deploy**. This should save the code.



Scroll up and click on the **+ Add Trigger** button. Select **AWS IoT** as the source and click **Custom IoT Rule** and under Rule click **Existing rules**. Choose the VentilationAuto rule. Click add and.



Add trigger

Trigger configuration [Info](#)

 **AWS IoT**
aws asynchronous devices iot

☒ Custom IoT rule 
☐ IoT Button

Rule
Pick an existing rule, or create a new one.
☐ Create a new rule
☒ Existing rules 

Rule description

Rule query statement
SELECT tvoc FROM 'esp32'

Lambda will add the necessary permissions for AWS IoT to invoke your Lambda function from this trigger. [Learn more](#)  about the Lambda permissions model.

Head over to the configuration tab inside of the function and click on **Permissions**. Click on the hyperlink **Role name**. This will open a new tab in the IAM section.

AutoVentilation

Function overview [Info](#)

Diagram | Template

 **AutoVentilation**
Layers (0)

 **AWS IoT**
[+ Add trigger](#)

Code | Test | Monitor | **Configuration** | Aliases | Versions

General configuration
Triggers
Permissions 
Destinations
Function URL
Environment variables
Tags
VPC

Execution role
Role name
[AutoVentilation-role-ii4z25rl](#) 
Resource summary
To view the resources and actions that your function has permission to access, choose a resource.

 **AWS IoT**
1 action, 1 resource

By action

By resource

In the IAM console, click **Add Permissions** and **Attach policies**. Search for “AWSIoTFullAccess”. Click the box next to it and scroll down to click on the **Add permissions** button. It should look like the image below. Now, AWS will automatically turn the fan on when the TVOC readings are above 500 ppb.

AutoVentilation-role-ii4z25rl [Info](#)

Summary

Creation date

April 23, 2025, 17:14 (UTC-04:00)

Last activity

✓ 13 hours ago

Permissions

Trust relationships

Tags

Last Accessed

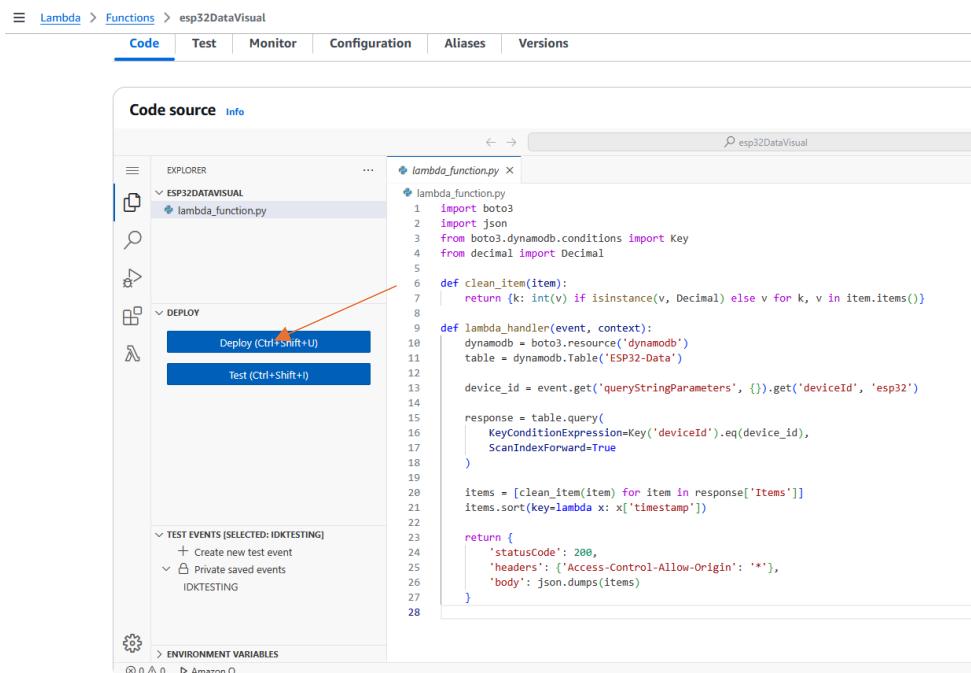
Revoke sessions

Permissions policies (2) [Info](#)

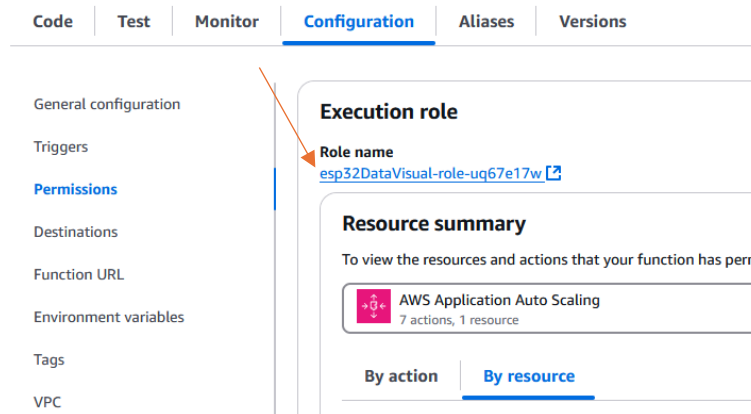
You can attach up to 10 managed policies.

Search		Filter by Type
		All types
☐	Policy name ✕	Type
☐	AWSIoTFullAccess	AWS managed
☐	AWSLambdaBasicExecutionRole-543aa3f8-d7f2-4ee2-8ef4-220...	Customer managed

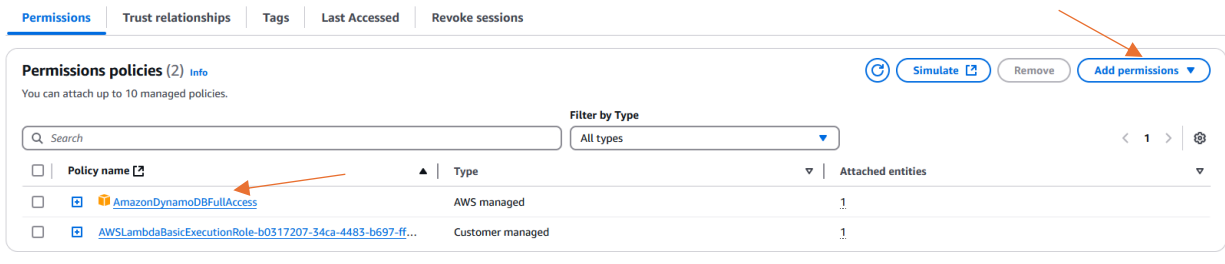
5) **PART 5; DASHBOARD IMPLEMENTATION.** Now, with the rules section completed, we need to create the actual dashboard. Head to the Lambda service in AWS and in the top right click on **Create function**. Choose **Author from scratch**, enter the name “esp32DataVisual” and choose the **Python** runtime. Choose **x86_64** as the architecture. This is the same setup as the AutoVentilation, except for the name. Scroll down to the code source section and paste the esp32DataVisualScript.py provided from the GitHub. Click deploy like last time and it should save the code.



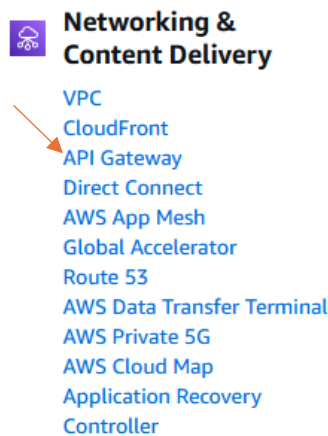
Click on the configuration menu and you should see the **Role name** and a link under it.



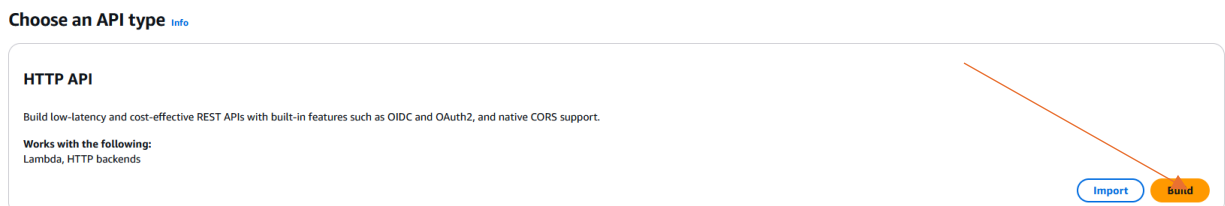
Click on that link and it will bring you to the IAM role associated with this function. Click on **Add permissions -> Attach policies** and search for **AmazonDynamoDBFullAccess**. This will give the function the ability to grab the data from the table.



Next, go back to the AWS homepage and go to the AWS API Gateway service.



Click **Create API** in the top right and choose **HTTP API**.



Enter the name as “ESP32DataAPI”, choose IPv4 as the IP address type, and add an integration. Choose **Lambda** under the integration and choose the lambda function **esp32DataVisual**. Click next.

Configure API

API details

API name
An HTTP API must have a name. The name is a non-unique value you use to identify and organize your APIs. To programmatically refer to this API, use the API ID that API Gateway generates for you.
ESP32DataAPI

IP address type [Info](#)
Select the type of IP addresses that can invoke the default endpoint for your API. You don't need to redeploy your API for the update to take effect.
☒ **IPv4**
Includes only IPv4 addresses.
☐ **Dualstack**
Includes IPv4 and IPv6 addresses.

Integrations (1) [Info](#)
Specify the backend services that your API will communicate with. These are called integrations. For a Lambda integration, API Gateway invokes the Lambda function and responds with the response from the function. For an HTTP integration, API Gateway sends the request to the URL that you specify and returns the response from the URL.

Lambda [Remove](#)

AWS Region us-east-2 **Lambda function** Q am:aws:lambda:us-east-2:942709711968:function:esp32DataVisual X **Version** 2.0 [Learn more](#)

[Add integration](#)

[Cancel](#) [Review and create](#) [Next](#)

The configure routes tab is next, but everything should be in place already. Click next.

Configure routes - optional

Configure routes [Info](#)

API Gateway uses routes to expose integrations to consumers of your API. Routes for HTTP APIs consist of two parts: an HTTP method and a resource path (e.g., GET /pets). You can define specific HTTP methods for your integration (GET, POST, PUT, PATCH, HEAD, OPTIONS, and DELETE) or use the ANY method to match all methods that you haven't defined on a given resource.

Method ANY **Resource path** /esp32DataVisual **Integration target** esp32DataVisual [Remove](#)

[Add route](#)

[Cancel](#) [Review and create](#) [Previous](#) [Next](#)

Under the review and create step click create to create the API. It should look similar to the image below.

Review and create

API name and integrations

Edit

API name

ESP32DataAPI

IP address type

IPv4

Integrations

- esp32DataVisual (Lambda)

Routes

Edit

Routes

- ANY /esp32DataVisual → esp32DataVisual (Lambda)

Stages

Edit

Stages

- \$default (Auto-deploy: enabled)

Cancel

Previous

Create

With the API created head back to the Lambda section of AWS and choose the “esp32DataVisual” function. Like last time, click add trigger. The source this time will be **API Gateway**. Click **Use existing API**, and then choose the API we just created. Use **\$default** as the Deployment stage and **Open** as the Security. Click **Add** and now the API should be connected to the Lambda function.

Add trigger

Trigger configuration info

API Gateway

aws api application-services backend HTTP REST serverless

Add an API to your Lambda function to create an HTTP endpoint that invokes your function. API Gateway supports REST, HTTP, and WebSocket APIs. [Learn more](#)

Intent

Use an existing api or have us create one for you.

Create a new API

☒ Use existing API

Existing API

When you attach the trigger to your existing API, API Gateway creates a new resource path based on your Lambda function's name and adds the ANY method that invokes your Lambda function.

4a2reb6022

X

Deployment stage

Select the API stage where Lambda adds the trigger. You can use the deployment history in the API Gateway console to revert or change the active deployment of this stage.

\$default

⌂

⚠ When you attach the trigger to your API, API Gateway automatically deploys your API to your deployment stage. Any other updates to your API resources are deployed when you attach the trigger.

Security

Configure the security mechanism for your API endpoint.

Open

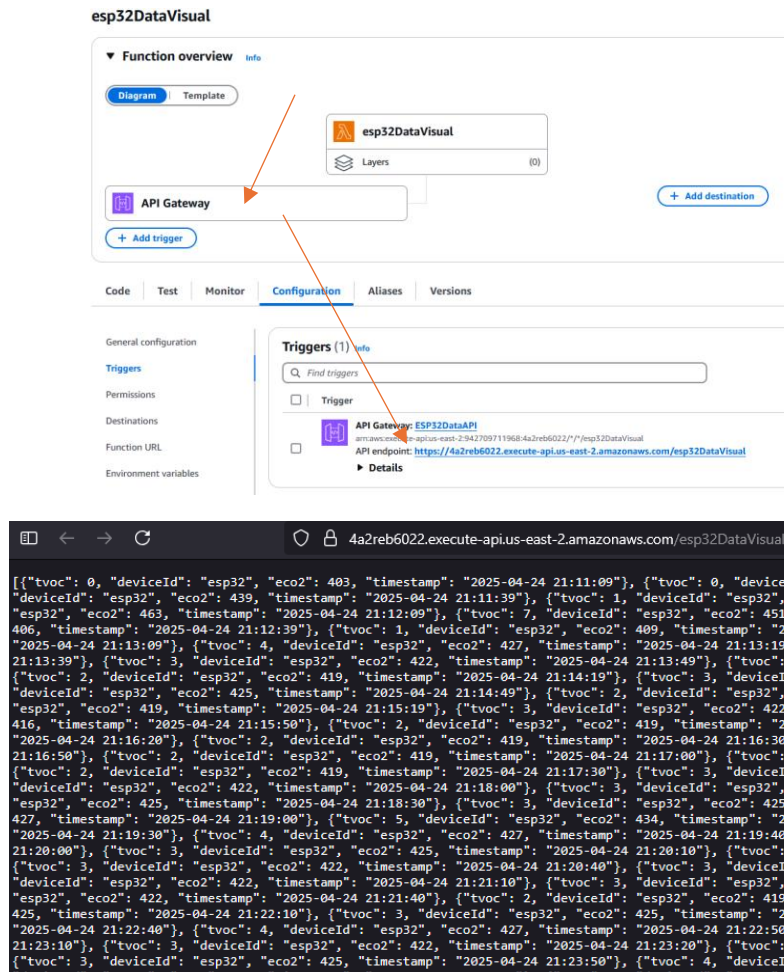
⌵

Lambda will add the necessary permissions for Amazon API Gateway to invoke your Lambda function from this trigger. [Learn more](#) about the Lambda permissions model.

Cancel

Add

With the API connected, click on the **API Gateway** trigger and it will send you to the configuration menu. Click on the **API endpoint**, which will send you to the AWS hosted API website. It should look similar to the images below.



With the API pulling the DynamoDB table data correctly the last step is to fancy up the webpage to show it as a graph. Download the Dashboard.html file provided in the GitHub and fill in your personal API endpoint in the code. Save the file anywhere on your computer with this modification and make sure to retain the .html file type.

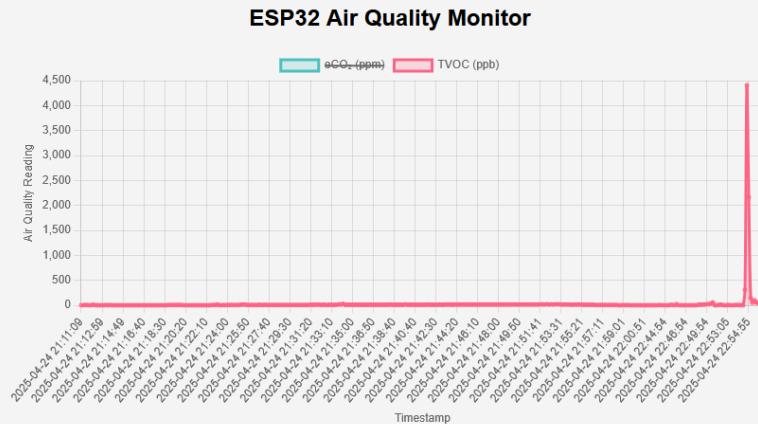
```
<script>
  let chart;

  // This function uses the API we created to grab the DynamoDB
  async function fetchData() {
    const response = await fetch("https://youramazonAPIhere.amazonaws.com/esp32DataVisual");
    const data = await response.json();

    const timestamps = data.map(item => item.timestamp);
    const eco2 = data.map(item => item.eco2);
    const tvoc = data.map(item => item.tvoc);

    return { timestamps, eco2, tvoc };
  }
```

After that, open the html file using any web browser and it should bring you to a graph of the last 30 minutes of pulled DynamoDB sensor data.



This takes us to the end of the implementation. You should now have a full system that takes in the TVOC levels, sends it to AWS, and gives you a live graph of that data.

Source Code & Documentation:

This our GitHub repository for all the scripts used in this project:

<https://github.com/EKarnes825/IoTProj>

Lessons Learned & Reflection:

This project really pushed us as a team to learn outside of the classroom setting. From the very beginning, we wanted to base our project on implementing a home device that measures the level of TVOCs in the home. After our research process, we realized that to encourage the implementation of air quality monitors in the home, it must become affordable. We settled on using the ESP32 microcontroller because it was cheaper compared to the Raspberry Pi but still provided everything we needed. Because we opted to use another microcontroller that's not a Raspberry Pi, we didn't know how to use the AWS interface with that microcontroller. However, there was not a huge difference except how to use the certificates and the code. There are lots of handy packages and documents on how to get MQTT working with Arduino. Another hardship was that the code was entirely in C++. We adapted the code given in the labs into C++ as best we could and got them working using those imported packages. There were hurdles that weren't covered in the lab due to the hardware used so we had to spend more time researching how to solve these issues. The biggest issue was implementing the dashboard requirement as we didn't know how to make a line chart using the collected data. This took us several hours to figure out, but we eventually got it working. Overall, this was a good assignment to end the semester by using everything we learned to create a working IoT device with full capabilities.